

Redes neurais recorrentes (RNNs) e processamento de linguagem natural (NLP)

Segundo Geron (2019), as redes neurais recorrentes (recurrent neural networks – RNNs) e o processamento de linguagem natural (natural language processing – NLP) são elementos muito importantes em deep learning, inspirados na capacidade de processar sequências e compreender linguagem humana.

Devemos destacar que as redes neurais recorrentes e o processamento de linguagem natural são ferramentas muito úteis para resolver problemas complexos, podendo ser utilizadas na previsão de séries temporais e até mesmo na tradução automática e análise de texto, integrando-se às tecnologias modernas de inteligência artificial.

Aprenderemos os fundamentos das redes neurais recorrentes e o processamento de linguagem natural, começando por sua estrutura básica, incluindo o conceito de recorrência e estados ocultos. Também veremos as variações avançadas, como LSTMs (Long Short Term Memory) e GRUs (Gated Recurrent Unit), e os principais componentes do processamento de linguagem natural, como tokenização e embeddings.

Se faz necessário analisar os desafios associados ao treinamento de das redes neurais recorrentes, como gradientes evanescentes, e as estratégias para superá-los. Por fim, será importante destacar algumas ferramentas práticas, como spaCy, NLTK e Hugging Face Transformers, que facilitam o desenvolvimento e a implementação de modelos avançados, popularizando o acesso a soluções de redes neurais recorrentes e o processamento de linguagem natural.

1 Fundamentos de RNNs

As redes neurais recorrentes e o processamento de linguagem natural são mecanismos do deep learning, que impulsionam alguns avanços significativos na interação entre humanos e máquinas, conforme nos ensina Geron (2019).

Para Suave (2024), as redes neurais recorrentes são arquiteturas especializadas projetadas para processar dados sequenciais, como textos, séries temporais e sinais de áudio, enquanto o processamento de linguagem natural engloba um conjunto amplo de técnicas e algoritmos voltados para a compreensão, geração e manipulação de linguagem humana, frequentemente utilizando redes neurais recorrentes, entre outros modelos.

Como sabemos, as redes neurais recorrentes são projetadas para processar dados sequenciais, utilizando a recorrência para preservar informações de passos anteriores, o que as torna ideais para aplicações como previsão de séries temporais. A partir de Geron (2019), veremos suas propriedades fundamentais, limitações e aplicações práticas, com ênfase em sua relevância.

1.1 Estrutura das RNNs e o conceito de recorrência

Segundo Geron (2019), podemos dizer que as redes neurais recorrentes distinguem-se das redes neurais tradicionais por sua capacidade de processar sequências, utilizando um estado oculto que é atualizado iterativamente ao longo do tempo. Em cada passo temporal t , a saída de um neurônio é usada como entrada para o próximo passo $t+1$, permitindo que a rede mantenha um histórico de informações anteriores, como uma memória.

Matematicamente, para uma entrada x_t no tempo t , o estado oculto h_t é calculado como:

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

Em que w_h e w_x são matrizes de pesos, b é o viés e f é uma função de ativação. Geralmente essa estrutura recorrente permite que as redes neurais recorrentes capturem dependências temporais; mas esse cenário apresenta desafios, como gradientes evanescentes ou explosivos, que dificultam o aprendizado em sequências longas.



IMPORTANTE

Para mitigar esses problemas ou amenizar maiores desafios, é recomendada a aplicação de técnicas de gradiente clipping e arquiteturas avançadas, como LSTMs e GRUs. Veremos mais sobre esses desenvolvimentos em capítulos posteriores.

1.2 Aplicações em séries temporais, análise de texto e NLP

As redes neurais recorrentes, segundo Suave (2024), têm o que podemos chamar de laços de realimentação, ou seja, uma saída de rede pode influenciar suas próprias entradas em momentos subsequentes. Vamos aprofundar o assunto na seção seguinte.

Já ficou claro que as redes neurais recorrentes são amplamente aplicadas em tarefas que envolvem sequências. Em séries temporais, elas são usadas para prever valores futuros, como preços de ações, padrões climáticos ou consumo energético, com base em dados históricos, segundo Silveira e Bullock (2017).

Por exemplo, as redes neurais recorrentes podem analisar dados diários de temperatura para prever condições futuras. Já em um processamento de linguagem natural, as redes neurais recorrentes são empregadas em tarefas como classificação de sentimentos, geração de texto e reconhecimento de fala.

Um exemplo prático é a análise de sentimentos, em que uma rede neural recorrente processa a sequência de palavras em uma frase como “Esse filme é incrível!” para determinar sua polaridade positiva. Além disso, as redes neurais recorrentes são úteis em tarefas como tradução automática, modelagem de linguagem e até geração de música, em que a ordem dos elementos é essencial.

Em alguns estudos não tão antigos, as redes neurais recorrentes também têm sido combinadas com técnicas de atenção para melhorar o desempenho em tarefas complexas, como resumo de textos longos.

2 Redes LSTM e GRU

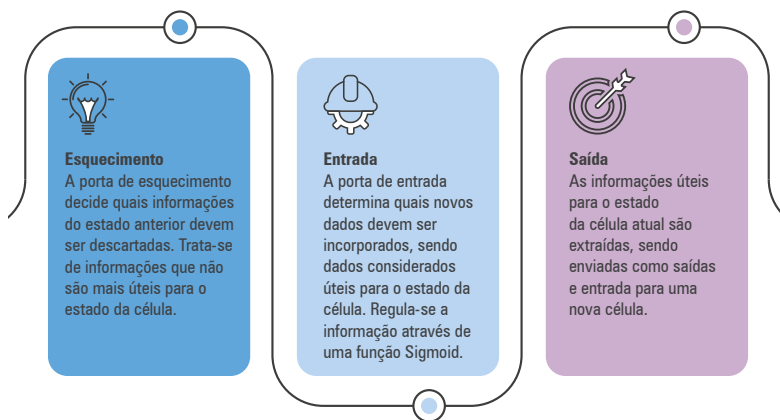
Para Geron (2019), as limitações das redes neurais recorrentes tradicionais, como os problemas de gradientes evanescentes, levaram ao desenvolvimento de arquiteturas mais robustas, como as LSTMs e GRUs, que lidam melhor com dependências de longo prazo. Por isso, precisamos explorar essas arquiteturas, suas estruturas, vantagens e aplicações práticas, além de discutir cenários em que cada uma é mais adequada.

2.1 Introdução às LSTM (long short-term memory)

As redes neurais recorrentes têm a característica conhecida de sofrerem de memória de curto prazo. Imaginando a existência de uma sequência longa o suficiente, as redes recorrentes terão dificuldade em transportar informações das etapas anteriores para as posteriores.

Geron (2019) nos mostra que as LSTMs foram projetadas para capturar dependências de longo prazo em sequências, superando as limitações das redes neurais recorrentes tradicionais. Elas introduzem uma célula de memória e três portas principais: entrada (input gate), esquecimento (forget gate) e saída (output gate).

Figura 1 – Etapas das LSTMs



Fonte: adaptado de Geron (2019).

A equação para a célula de memória C_t é:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{c}_t$$

Em que f_t é a porta de esquecimento, i_t é a porta de entrada e $\tilde{c}_t$ é o candidato a novo estado.

As LSTMs, como sabemos, são particularmente eficazes em tarefas como tradução automática, em que o contexto de frases longas precisa ser mantido, e em geração de texto, em que a coerência ao longo de parágrafos é crucial. Por exemplo, ao traduzir uma frase complexa como “Embora o dia estivesse ensolarado, ele decidiu ficar em casa”, a LSTM mantém o contexto inicial para garantir uma tradução precisa.



PARA PENSAR

Com base no que já vimos aqui, você consegue imaginar mais possibilidades de uso das LSTMs? Você acredita que elas podem ser eficiente para legendas de imagens, geração de texto ou até mesmo para chatbots?

2.2 Características das GRUs (gated recurrent units)

Conforme Geron (2019), as GRUs são uma alternativa simplificada às LSTMs, com menos parâmetros, mas desempenho comparável em muitas tarefas. Elas combinam as portas de entrada e esquecimento em uma única porta de atualização e utilizam uma porta de reset para controlar a influência do estado anterior.

A atualização do estado oculto em uma GRU é dada por:

$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t$$

Em que z_t é a porta de atualização e $\tilde{h}_t$ é o candidato a novo estado. As GRUs são computacionalmente mais eficientes e, portanto, ideais para aplicações com recursos limitados, como dispositivos móveis ou em tarefas como previsão de séries temporais e geração de texto. Por exemplo, uma GRU pode ser usada em um chatbot para gerar respostas rápidas e coerentes com base em diálogos anteriores, exigindo menos poder computacional que uma LSTM.

Assim como as portas das LSTMs, as portas na GRU são desenvolvidas para filtrar de maneira seletiva qualquer informação irrelevante, mantendo assim apenas o que é útil. Essas portas são, em sua essência, vetores contendo valores entre 0 e 1 que serão multiplicados com os dados de entrada e, quando existente, de estado oculto.

Um valor '0(zero)' nos vetores indica que os dados correspondentes no estado de entrada ou oculto não são importantes; assim, podemos dizer que vão retornar como zero. Por um outro ponto de vista, um valor 1 no vetor significa que os dados correspondentes são importantes e serão usados.

Elaborando de forma mais objetiva uma comparação entre as LSTMs e as GRUs: enquanto as LSTMs são mais expressivas devido à sua estrutura complexa, as GRUs oferecem uma alternativa mais leve, com treinamento mais rápido e menor consumo de memória. A escolha entre elas depende do caso de uso: LSTMs são preferidas em tarefas que exigem memória de longo prazo, como tradução de textos longos, enquanto GRUs são mais adequadas para aplicações em tempo real ou com restrições computacionais.

Ambas superam as redes neurais recorrentes tradicionais em tarefas de processamento de linguagem natural e séries temporais, mas podem ser substituídas por modelos transformadores em aplicações mais modernas. Veremos mais sobre esse assunto em seções posteriores.

3 Modelos para processamento de linguagem natural (NLP)

Seguindo os ensinamentos de Suave (2024), o processamento de linguagem natural abrange um conjunto de técnicas para interpretar, gerar e manipular linguagem humana, frequentemente utilizando RNNs, LSTMs, GRUs e, mais recentemente, modelos baseados em transformadores.

Veremos de maneira detalhada os principais componentes do processamento de linguagem natural, incluindo tokenização, embeddings, modelos seq2seq e a ascensão dos transformadores, além de exemplos práticos de aplicação.

3.2.1 Tokenização e embeddings de palavras (Word2Vec, GloVe)

Segundo Suave (2024), a tokenização é o processo inicial de dividir texto em unidades menores, como palavras, subpalavras ou caracteres, para que possam ser processados por modelos de processamento de linguagem natural. O autor faz uma analogia sobre a tokenização, em que a compara com uma preparação de ingredientes antes de cozinhar.

Segundo Geron (2019), após a tokenização, as palavras são convertidas em representações numéricas chamadas embeddings, que capturam relações semânticas entre palavras. Modelos como Word2Vec e GloVe são amplamente utilizados para gerar embeddings.

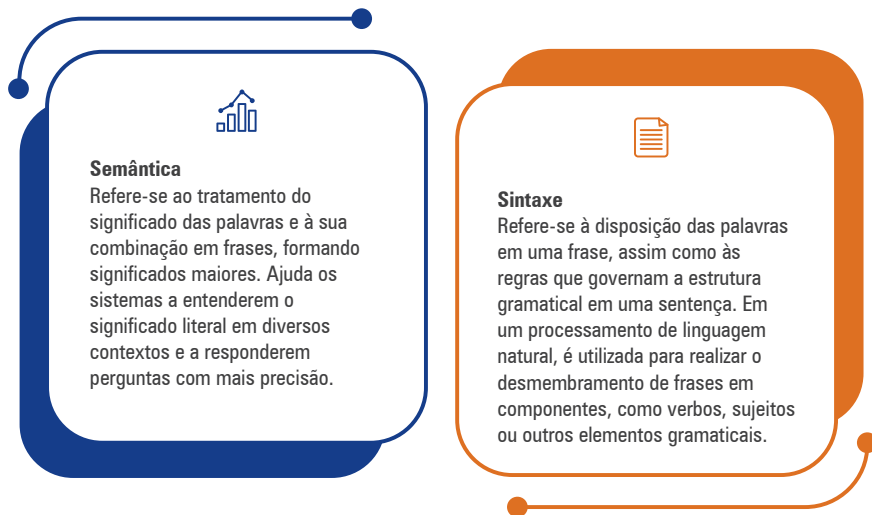
O Word2Vec usa duas abordagens: CBOW (continuous bag of words), que prevê uma palavra com base em seu contexto, e Skip-gram, que prevê o contexto a partir de uma palavra. O GloVe, por outro lado, utiliza concorrências globais de palavras em um formato para criar vetores densos.

Por exemplo, os embeddings de “rei” e “rainha” em Word2Vec são próximos devido à sua semelhança semântica. Esses embeddings servem como entrada para redes neurais em tarefas como classificação de texto, análise de sentimentos e tradução automática.

Em algumas comparações, embeddings contextuais, como os gerados por BERT (bidirectional encoder representations from transformers), têm superado abordagens estáticas como Word2Vec, oferecendo representações dinâmicas baseadas no contexto da frase.

Para encerrarmos, é interessante reforçarmos alguns conceitos, como o de semântica e sintaxe, conforme Suave (2019) nos ensina.

Figura 2 – Semântica e sintaxe



Fonte: adaptado de Suave (2019).

3.2.2 Redes seq2seq e tradução automática

Vejamos a seguir as redes seq2seq (sequence-to-sequence), modelos que transformam uma sequência de entrada em uma sequência de saída, amplamente usadas em tradução automática, resumo de texto e chatbots (Mueller; Massaron, 2018).

Elas consistem em um codificador que converte a sequência de entrada. Podemos citar como exemplo uma frase em inglês, que aqui é um vetor de contexto, e um decodificador, que gera a sequência de saída, que, nesse exemplo, seria uma frase em português.

Frequentemente, LSTMs ou GRUs são usadas em ambas as partes, com mecanismos de atenção para focar em partes relevantes da entrada. Por exemplo, ao traduzir “I love to travel” para “Eu amo viajar”, o mecanismo de atenção garante que a palavra “love” seja corretamente associada a “amo”.

A introdução de modelos baseados em atenção, como os transformadores, tem sofrido grandes avanços quando relacionados às redes seq2seq, oferecendo, em muitos casos, maior precisão e eficiência em tarefas complexas.

Sobre os transformadores, já fazem alguns anos que os modelos baseados neles, como BERT, GPT (generative pre-trained transformer) e T5 (text-to-text transfer transformer), têm substituído as redes neurais recorrentes em muitas aplicações de processamento de linguagem natural devido à sua capacidade de processar sequências em paralelo e capturar dependências bidirecionais (Geron, 2019).

Diferentemente das redes neurais recorrentes, que processam sequências de forma sequencial, os transformadores utilizam mecanismos de autoatenção para pesar a importância de cada palavra em uma frase, independentemente de sua posição.

Por exemplo, o BERT pode entender o contexto de uma palavra ao considerar todas as palavras da frase simultaneamente, o que o torna ideal para tarefas como resposta a perguntas e classificação de texto. Embora os transformadores sejam computacionalmente mais exigentes, sua eficiência em grandes datasets e desempenho superior os tornaram o padrão em aplicações modernas de processamento de linguagem natural.

4 Ferramentas para NLP e RNNs

Para Geron (2019), ferramentas específicas para processamento de linguagem natural e redes neurais recorrentes simplificam a implementação de modelos complexos, desde o pré-processamento de texto até a utilização de embeddings pré-treinados e treinamento de redes. Por isso, é interessante apresentar bibliotecas populares, exemplos práticos e discutir frameworks adicionais que complementam spaCy e NLTK (natural language toolkit).

4.1 Introdução ao spaCy e NLTK

Segundo Mueller e Massaron (2018), as bibliotecas spaCy e NLTK são amplamente usadas em processamento de linguagem natural. O spaCy é otimizado para desempenho em aplicações industriais, oferecendo tokenização, análise sintática, reconhecimento de entidades nomeadas e suporte a múltiplos idiomas.

Por exemplo, o spaCy pode identificar “São Paulo” como uma entidade geográfica em um texto. O NLTK, com uma abordagem mais acadêmica, é ideal para experimentação, fornecendo ferramentas para tokenização, stemming, lematização e análise de sentimentos, para interpretar e analisar o tom emocional de textos, áudios ou outras interações.

Um exemplo de código de tokenização com spaCy em Python é:

```
import spacy
nlp = spacy.load("en_core_web_sm")
texto = "Artificial intelligence is transforming the world."
doc = nlp(texto)
tokens = [token.text for token in doc]
print(tokens) # Saída: ['Artificial', 'intelligence', 'is',
'transforming', 'the', 'world', '.']
```

O uso do spaCy é mais indicado em pipelines industriais devido à sua eficiência, enquanto o NLTK é amplamente utilizado em ensino e pesquisa devido à sua flexibilidade. Além disso, ferramentas como Hugging Face Transformers complementam o spaCy e o NLTK, oferecendo acesso a modelos pré-treinados, como o BERT.

4.1.1 Implementação de embeddings pré-treinados

Conforme Geron (2019), embeddings pré-treinados, como GloVe e Word2Vec, melhoram o desempenho de modelos de processamento de linguagem natural ao fornecer representações de palavras aprendidas em grandes blocos de dados.

Um exemplo prático de código Python com Keras usando embeddings GloVe é:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
import numpy as np
# Carregando embeddings GloVe (exemplo simplificado)
embedding_dim = 100
embedding_matrix = np.random.rand(10000, embedding_dim) #
# Substitua por GloVe real
# Criando o modelo
model = Sequential([
    Embedding(10000, embedding_dim, weights=[embedding_
matrix], input_length=50, trainable=False),
    LSTM(64, return_sequences=False),
    Dense(1, activation='sigmoid')
])
# Compilando o modelo
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
model.summary()
```

Nesse exemplo, a camada embedding utiliza uma matriz GloVe pré-treinada, seguida por uma camada LSTM para processar sequências de texto e uma camada densa para classificação binária, como a análise de sentimentos. Para embeddings contextuais, frameworks como Hugging Face Transformers permitem carregar modelos como BERT, que podem ser fine-tuned para tarefas específicas, ou seja, um ajuste fino que permite adaptar modelos pré-treinados.

Podemos citar algumas outras ferramentas que podem ser muito úteis, a depender da aplicabilidade. Além de spaCy e NLTK, frameworks como TensorFlow, PyTorch e Hugging Face Transformers são amplamente utilizados para implementar modelos de processamento de linguagem natural e redes neurais recorrentes.

O TensorFlow e o PyTorch oferecem suporte nativo para construir e treinar LSTMs e GRUs, enquanto o Hugging Face Transformers fornece

acesso a modelos pré-treinados de última geração, como BERT e GPT, que podem ser integrados com embeddings pré-treinados ou fine-tuned para tarefas específicas.

Por exemplo, a utilização do Hugging Face permite carregar um modelo BERT para classificação de texto com poucas linhas de código, simplificando o desenvolvimento de aplicações avançadas.

Considerações finais

Neste capítulo, vimos características relacionadas entre as redes neurais recorrentes e o processamento de linguagem natural, destacando suas contribuições fundamentais para a inteligência artificial e deep learning.

As redes neurais recorrentes, com sua capacidade de processar dados sequenciais por meio de estados ocultos e recorrência, revelam-se ferramentas poderosas para aplicações como previsão de séries temporais e análise de texto, apesar de desafios como gradientes evanescentes.

Vimos um pouco sobre algumas ferramentas como spaCy, NLTK e Hugging Face Transformers e como elas podem popularizar o acesso ao desenvolvimento de modelos de redes neurais recorrentes e o processamento de linguagem natural.

Vimos que a ascensão dos modelos baseados em transformadores, como BERT e GPT, representa uma evolução significativa, superando as redes neurais recorrentes tradicionais em muitas tarefas de processamento de linguagem natural devido à sua capacidade de processar sequências em paralelo e capturar contextos bidirecionais, embora exijam maior capacidade computacional.

Por fim, o capítulo reforçou a importância de compreender os fundamentos teóricos e práticos de RNNs e LNP, bem como sua implementação, para que profissionais e pesquisadores possam contribuir para o desenvolvimento de soluções inovadoras relacionadas à inteligência artificial.

Referências

GERON, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow:** Concepts, Tools, and Techniques to Build Intelligent Systems. [S. l.]: O'Reilly Media, 2019.

MUELLER, J. P.; MASSARON, L. **Artificial Intelligence For Dummies.** Hoboken: For Dummies, 2018.

SILVEIRA, G.; BULLOCK, B. **Machine learning:** introdução à classificação. São Paulo: Casa do Código, 2017.

SUAVE, A. A. **Inteligência artificial.** São Paulo: Editora Freitas Bastos, 2024.